

Уважаемый пользователь!

Обращаем ваше внимание, что система Антиплагиус отвечает на вопрос, является тот или иной фрагмент текста заимствованным или нет. Ответ на вопрос, является ли заимствованный фрагмент именно плагиатом, а не законной цитатой, система оставляет на ваше усмотрение.

Отчет о проверке № 10067838

Дата загрузки: 2026-06-25 15:54:36

Пользователь: anon_VPbluTTpzx@antiplagius.ru, ID: 10067838

Отчет предоставлен сервисом «Антиплагиат»
на сайте antiplagius.ru/

Информация о документе

№ документа: 10067838

Имя исходного файла: CoffeeOchetdb.pdf

Размер файла: 0.50 МБ

Размер текста: 31279

Слов в тексте: 5037

Число предложений: 541

Информация об отчете

Дата: 2026-06-25 15:54:36 - Последний готовый отчет

Оценка оригинальности: 100%

Заемствования: 0%

100.00%

0%

Источники:

Доля в тексте

Ссылка

Информация о документе:

Частное учреждение профессионального образования "Высшая школа предпринимательства" (ЧУПО "ВШП") КУРСОВОЙ ПРОЕКТ "Разработка базы данных для кофейни" Выполнил: студент 3-го курса специальности 09.02.07 "Информационные системы и программирование" Чернаков Артём Дмитриевич подпись: _____ Проверил: преподаватель дисциплины, преподаватель ЧУПО "ВШП", к.ф.н. Ткачев П.С. оценка: _____ подпись: _____ Тверь, 2025 г. СОДЕРЖАНИЕ ВВЕДЕНИЕ 3 ГЛАВА 1. Теоретические основы разработки базы данных для кофейни 6 1.1 Характеристика предметной области кофейни 6 1.2 Обоснование необходимости автоматизации учёта 7 1.3 Обзор существующих программных решений 8 1.4 Реляционные базы данных и принципы их проектирования 9 Выводы по первой главе 10 ГЛАВА 2. Разработка базы данных для кофейни 11 2.1 Постановка задачи и требования к базе данных 11 2.2 Концептуальное и логическое проектирование 12 2.3 Нормализация и физическая модель данных 14 2.4 Создание таблиц и ограничений целостности 15 2.5 Программные объекты базы данных 17 2.6 Разграничение доступа к данным 20 2.7 Запросы к базе данных и тестирование 20 Выводы по второй главе 22 ЗАКЛЮЧЕНИЕ 23 СПИСОК ИСТОЧНИКОВ 24 ПРИЛОЖЕНИЕ А 25 ПРИЛОЖЕНИЕ Б 26 ВВЕДЕНИЕ Кофеен в любом городе сегодня очень много, и удержаться на этом рынке непросто. Гости привлекают не только вкусным кофе, но и скоростью обслуживания, а владельцу при этом важно понимать, что и когда продаётся, кто из сотрудников работает с клиентами и сколько денег приносит заведение. Пока поток небольшой, со всем этим справляется блокнот или таблица в Excel. Но стоит потоку вырасти, как бумажный учет начинает подводить: цифры путаются, данные теряются, а собрать отчёт за месяц превращается в отдельную задачу. Выход здесь один - перевести учёт в информационную систему, в основе которой лежит база данных. В ней сведения о меню, заказах, гостях, сотрудниках и платежах хранятся в одном месте и согласованы между собой, а сама СУБД следит за тем, чтобы в базу не попали явно ошибочные записи. Именно поэтому проектирование базы данных я считаю центральной задачей при автоматизации кофейни: всё остальное - касса, мобильное приложение, отчёты - строится уже поверх неё. Актуальность темы в том, что от качества схемы данных напрямую зависит, насколько удобно и быстро потом будет работать вся система. Просчеты, допущенные на этапе проектирования, обходятся дорого: переделывать структуру действующей базы куда сложнее, чем продумать её заранее. Поэтому умение проектировать базы данных входит в число базовых навыков специалиста по информационным системам. Цель работы - спроектировать и реализовать реляционную базу данных для учёта основных процессов кофейни. Для достижения этой цели поставлены следующие задачи: 1. изучить предметную область и выделить основные объекты учёта кофейни; 2. рассмотреть теоретические основы реляционных баз данных и принципы их проектирования; 3. проанализировать существующие системы автоматизации общественного питания; 4. сформулировать

требования к базе данных; 5. спроектировать базу данных на концептуальном, логическом и физическом уровнях и привести её к третьей нормальной форме; 6. реализовать структуру в MySQL: таблицы, ограничения целостности, представление, хранимую процедуру, функцию, триггер, транзакцию и роли пользователей; 7. разработать типовые запросы и протестировать работу базы данных. Объект исследования - процессы работы кофейни как предприятия общественного питания. Предмет исследования - структура реляционной базы данных, в которой эти процессы отражаются. В работе применялись методы системного анализа предметной области, моделирования "сущность - связь", теории нормализации, а также практические методы реализации структуры данных средствами языка SQL. Работа состоит из введения, двух глав, заключения, списка источников и приложений. Первая глава носит теоретический характер: в ней разобрана предметная область, выполнен обзор готовых решений и изложены основы реляционных баз данных. Вторая глава практическая - в ней описана собственная разработка базы данных в СУБД MySQL, включая программные объекты, разграничение доступа, запросы и тестирование. ГЛАВА 1. Теоретические основы разработки базы данных для кофейни В этой главе нужно разобраться, как устроена работа кофейни с точки зрения учёта, почему ручные способы перестают справляться и какие готовые системы уже есть на рынке. Кроме того, здесь излагается теоретический базис - понятия реляционной модели, нормализации и средств управления данными, - на который опирается практическая часть работы. 1.1 Характеристика предметной области кофейни Кофейня - это небольшое предприятие общепита, которое зарабатывает в первую очередь на кофе, а заодно продаёт чай, десерты, выпечку и лёгкие закуски. По организации работы она занимает промежуточное положение между рестораном с обслуживанием за столиками и точкой быстрого питания: гость может и присесть за столик, и взять напиток с собой. От этого зависит и то, как придётся вести учёт продаж. Чтобы понять, что именно должна хранить база, я сначала выписал объекты, вокруг которых крутится вся работа заведения. Получился такой список: ● позиции меню - перечень всех блюд и напитков, предлагаемых заведением, с указанием их наименования, описания, цены и категории; ● клиенты - посетители заведения, сведения о которых необходимы для ведения программ лояльности, оформления доставки и анализа спроса; ● сотрудники - персонал кофейни (бариста, официанты, администраторы), обслуживающий и обслуживаемый заказы; ● столики - посадочные места в зале заведения, имеющие определённую вместимость и статус занятости; ● заказы - факты обращения клиента, объединяющие набор заказанных позиций, обслуживающего сотрудника и занятый столик; ● позиции заказа - детализация каждого заказа с указанием конкретных блюд и их количества; ● платежи - сведения об оплате заказов с указанием суммы, даты и способа оплаты. Эти объекты связаны между собой вполне очевидным образом. Заказ кто-то принимает (сотрудник), для кого-то (клиента) и обычно за каким-то столиком. В заказ входит одна или несколько позиций меню, причём один и тот же латте попадает в сотни разных заказов. Когда гость расплатился, к заказу добавляется платёж. Собственно, эти связи я потом и переносил в схему базы. Если посмотреть на кофейню как на набор процессов, то главный из них - принять заказ и довести его до оплаты. Рядом идут вспомогательные: вести меню (добавить позицию, поменять цену, временно убрать то, что закончилось), следить за столиками и учитывать сотрудников. Все они работают с одними и теми же данными, поэтому держать эти данные в одном общем хранилище гораздо удобнее, чем в разрозненных таблицах. Важная деталь: почти всё здесь привязано ко времени. У заказа есть дата, у платежа тоже. Если это аккуратно сохранять, потом можно посчитать выручку за месяц, найти часы пик или понять, какие позиции берут чаще. То есть база нужна не только для сиюминутного учёта, но и как архив, по которому удобно считать статистику. 1.2 Обоснование необходимости автоматизации учёта В маленьких заведениях учёт часто ведут на бумаге или в Excel. Пока заказов мало, это терпимо, но с ростом потока минусы такого подхода вылезают один за другим: ● высокая вероятность ошибок при ручном вводе данных и расчёте сумм заказов; ● дублирование информации и отсутствие единого источника достоверных данных; ● сложность оперативного поиска нужных сведений в больших объёмах записей; ● отсутствие контроля целостности - например, возможность оформить заказ на несуществующую позицию меню; ● трудоёмкость формирования отчётности и анализа деятельности заведения; ● риск безвозвратной потери данных при повреждении бумажных или локальных носителей. Реляционная база снимает почти все эти проблемы. Данные вводятся один раз, а используются сколько угодно. Часть проверок СУБД берёт на себя: она просто не даст сохранить заказ на блюдо, которого нет в меню, а при удалении заказа сама подчистит его строки. То есть согласованность поддерживается автоматически, а не держится на внимательности кассира. Ещё один плюс - на одну и ту же базу можно повесить сразу несколько программ: кассу, приложение для заказов, складской учёт, аналитику. Все они обращаются к общему хранилищу, и данные у них всегда одинаковые. Поэтому база и считается ядром всей системы. 1.3 Обзор существующих программных решений Прежде чем проектировать что-то своё, я посмотрел, как устроены готовые программы для общепита. Изобретать структуру с нуля смысла нет - проще посмотреть, какие данные хранят проверенные системы. Коротко о тех, что встречаются чаще всего. iiko - большая система для ресторанов и сетей: склад, касса, персонал, финансы, всё в одном. Функций море, но и порог входа высокий - внедрение долгое, владение дорогое. Для маленькой кофейни это явно из пушки по воробьям. R-Keeper - один из старожилов российского рынка, закрывает весь цикл от приёма заказа до отчётов. Надёжная и распространённая, но снова упирается в деньги: лицензии и обучение персонала стоят прилично. Poster и похожие облачные кассы, наоборот, заточены под малый бизнес - как раз под кофейню. Простой интерфейс, работа с планшета, оплата по подписке. Удобно, но есть обратная сторона: ты зависишь от поставщика и почти не можешь менять структуру данных под себя. Сравнительный анализ рассмотренных решений приведён в таблице 1. Критерий iiko R-Keeper Poster Целевой сегмент Крупные сети Рестораны, сети Малый бизнес Модель развёртывания Локальная/облако Локальная Облако Стоимость владения Высокая Высокая Средняя Сложность внедрения Высокая Высокая Низкая Гибкость настройки Высокая Средняя Низкая Таблица 1 - Сравнение существующих систем автоматизации Вывод из обзора простой: готовые системы для маленькой кофейни обычно избыточны, дороги и плохо гнутся под конкретные нужды. Поэтому разработать свою базу вполне оправданно - она учтёт ровно то, что нужно, и её не страшно дорабатывать. При этом сам набор сущностей у всех систем примерно одинаковый (меню, заказы, клиенты, сотрудники, платежи), и именно его я взял за основу. 1.4 Реляционные базы данных и принципы их проектирования Прежде чем

проектировать конкретную базу, кратко изложу теоретический минимум, на котором строится вся практическая часть. База данных - это организованное хранилище взаимосвязанных данных, а управляет ими система управления базами данных (СУБД) - программа, которая отвечает за хранение, поиск, изменение данных и контроль их целостности. Наиболее распространена реляционная модель, предложенная Э. Коддом. В ней данные представлены в виде отношений - таблиц, состоящих из строк (записей) и столбцов (полей). Каждая строка описывает один объект, а первичный ключ - это поле или набор полей, по которым строку можно однозначно отличить от остальных. Связи между таблицами реализуются через внешние ключи: поле одной таблицы ссылается на первичный ключ другой. Для проектирования структуры применяют модель "сущность - связь" (ER-модель), которую предложил П. Чен. В ней выделяют три понятия: сущности - объекты предметной области, о которых хранят данные; атрибуты - их свойства; и связи между сущностями. Связи бывают трёх типов: один к одному, один ко многим и многие ко многим. Последний тип в реляционной модели напрямую не выражается и разбивается на две связи "один ко многим" через промежуточную (ассоциативную) сущность. Чтобы данные не дублировались и не возникало аномалий при их изменении, отношения приводят к нормальным формам. На практике обычно достаточно третьей нормальной формы (3НФ). Кратко суть первых трёх форм такова: ● 1НФ - все значения атомарны (неделимы), нет повторяющихся групп; ● 2НФ - отношение в 1НФ, и все неключевые поля зависят от первичного ключа целиком, а не от его части; ● 3НФ - отношение во 2НФ, и нет транзитивных зависимостей: неключевое поле не зависит от другого неключевого поля. Кроме хранения данных, современные реляционные СУБД позволяют размещать логику прямо в базе. Представления (VIEW) - это сохранённые запросы, которые выглядят как виртуальные таблицы. Хранимые процедуры и функции - это подпрограммы на встроенном процедурном языке, которые выполняются на стороне сервера. Триггеры автоматически срабатывают при изменении данных в таблице. Транзакции объединяют несколько операций в одно неделимое целое. Наконец, система ролей и прав доступа разграничивает, кто и какие действия может выполнять. Все эти средства применяются во второй главе. Выводы по первой главе В первой главе разобрана предметная область кофейни, выделены семь основных объектов учёта и показано, почему ручной учёт уступает базе данных. Обзор готовых систем подтвердил, что для небольшого заведения собственная база оправдана. Также изложены теоретические основы: реляционная модель, ER-модель, нормальные формы и программные средства СУБД. Этот материал служит фундаментом для практической разработки во второй главе. ГЛАВА 2. Разработка базы данных для кофейни Во второй главе решается основная задача работы - спроектировать и реализовать базу данных. Здесь последовательно описаны постановка задачи, проектирование схемы, создание таблиц и ограничений, программные объекты, разграничение доступа, а также запросы и тестирование готовой базы. 2.1 Постановка задачи и требования к базе данных На основании анализа предметной области и обзора готовых решений задачу можно сформулировать так: нужна реляционная база, которая хранит данные обо всех основных объектах кофейни и поддерживает обычные операции - оформить заказ и принять оплату. Что конкретно она должна уметь, я свёл в список требований. Функциональные требования: ● хранение полного перечня позиций меню с указанием категории, цены и признака доступности; ● ведение картотеки клиентов и сотрудников заведения; ● учёт столиков с контролем их вместимости и текущего статуса; ● регистрация заказов с привязкой к клиенту, сотруднику и столику; ● детализация состава каждого заказа с указанием количества позиций; ● фиксация платежей по заказам с указанием суммы и способа оплаты; ● возможность построения аналитических запросов о выручке, популярности позиций и загрузке заведения. Есть и требования, которые напрямую не про функции, но не менее важны. Целостность данных должна держаться на самой СУБД, а не на дисциплине пользователя. Схему нужно довести как минимум до третьей нормальной формы. Кодировка обязана нормально хранить русский текст. И, наконец, базу хотелось бы потом расширять, не ломая то, что уже есть. Отдельно учтены требования дисциплины МДК.11.01: база должна включать не менее пяти связанных таблиц, не менее двух ролей доступа, не менее пяти типовых запросов, а также транзакцию, представление, хранимую процедуру, пользовательскую функцию, триггер и обработчик исключений. Все эти объекты реализованы и описаны далее в данной главе. 2.2 Концептуальное и логическое проектирование Проектирование я вёл в три захода: сначала концептуальная модель, потом логическая, затем физическая. На концептуальном уровне СУБД ещё не важна - здесь описывается, какие в предметной области есть объекты и как они связаны. Применительно к кофейне из анализа получилось семь сущностей: "Меню", "Клиенты", "Сотрудники", "Столики", "Заказы", "Детали заказа" и "Платежи". Между ними установлены такие связи: клиент - заказы, сотрудник - заказы и столик - заказы (все "один ко многим"); заказ - детали заказа ("один ко многим"); меню - детали заказа ("один ко многим"); заказ - платёж ("один к одному"). Здесь есть тонкий момент. Связь "Заказ - Меню" на самом деле "многие ко многим": в заказе много позиций, а каждая позиция встречается во многих заказах. Напрямую такое не выразить, поэтому её разбивают через промежуточную сущность "Детали заказа", которая хранит, какая позиция в каком заказе вошла и в каком количестве. Приём стандартный, его применяют почти в любой базе с подобными связями. На логическом уровне каждая сущность превратилась в таблицу, её свойства - в столбцы, а связи держатся на первичных и внешних ключах. Схему я нарисовал в MySQL Workbench, она показана на рисунке 1. Рисунок 1 - ER-диаграмма базы данных кофейни Всего таблиц получилось семь. Кратко поясню назначение каждой: customers - клиенты заведения: имя, телефон, e-mail, адрес доставки. Телефон и почту я пометил уникальными, чтобы один номер не оказался у двух гостей. employees - сотрудники: имя, должность, телефон и дата приёма на работу. tables - столики в зале: вместимость и текущий статус (свободен, занят, забронирован). menu - позиции меню: название, описание, цена, категория и флажок доступности, которым закончившуюся позицию можно временно спрятать, не удаляя её. customerorders - заказы, центральная таблица схемы; через внешние ключи ссылается на клиента, сотрудника и столик, хранит дату, статус и итоговую сумму. orderdetails - промежуточная таблица между заказами и меню: одна строка это одна позиция в одном заказе плюс количество. payments - платежи: ссылка на заказ, дата, сумма и способ оплаты; ссылка уникальна, поэтому связь с заказом строго один к одному. Таблица Назначение Первичный ключ customers Сведения о клиентах заведения CustomerID employees Сведения о сотрудниках EmployeeID tables Столики в зале заведения TableID menu Позиции меню MenuID customerorders Заказы клиентов OrderID orderdetails Состав (детали) заказов OrderDetailID payments Платежи по заказам PaymentID Таблица 2 - Состав

таблиц базы данных 2.3 Нормализация и физическая модель данных Нормализация - это приведение таблиц к виду, при котором данные не дублируются лишней раз и не возникает аномалий при вставке, изменении и удалении записей. До третьей нормальной формы схему я довёл осознанно, по шагам. Первая нормальная форма выполнена: нигде нет полей вроде "список блюд через запятую", состав заказа вынесен в orderdetails, и каждая позиция - отдельная строка. Вторая форма тоже соблюдена: первичные ключи во всех таблицах простые (один столбец с автоинкрементом), поэтому частичных зависимостей быть не может. Для третьей формы важно отсутствие транзитивных зависимостей: в таблице заказов лежит только CustomerID, а имя и телефон клиента хранятся в customers и не дублируются. Так же вынесены данные о сотруднике, столике и блюдах. Значит, схема в 3НФ. Единственное осознанное отступление - поле TotalAmount с уже посчитанной суммой заказа. Строго говоря, его можно вычислять на лету, но хранить готовый итог удобнее для отчётов; ниже описан триггер, который поддерживает это поле в актуальном состоянии автоматически. На физическом уровне схема привязана к MySQL 8.0. Эту СУБД я выбрал потому, что она бесплатная и с открытым кодом, с движком InnoDB поддерживает транзакции и внешние ключи, имеет подробную документацию и удобный визуальный редактор MySQL Workbench, а также одинаково подходит и для маленьких, и для крупных проектов. Кодировку задал utf8mb4 - она без проблем хранит русский текст. Типы данных я подбирал под смысл поля. Идентификаторы - INT с AUTO_INCREMENT. Цены и суммы - DECIMAL: для денег это принципиально, числа с плавающей точкой дают ошибки округления. Там, где вариантов немного (категория блюда, статус заказа, способ оплаты), удобен ENUM. Короткие строки - VARCHAR, длинные описания - TEXT, даты - DATE и DATETIME. Сводка типов приведена в таблице 3. Тип данных Назначение Пример столбца INT AUTO_INCREMENT Первичные ключи (идентификаторы) CustomerID, OrderID VARCHAR(n) Короткие строки FullName, Phone TEXT Длинные текстовые описания Description, Address DECIMAL(p,s) Дене